Aproximación de Funciones mediante Redes de Base Radial (RBF)

Informe Técnico de Implementación y Resultados

Autor: Gregory Uzcategui / CI: 27.376.191

Stack tecnológico: C++17 (Eigen 3.4), CMake, Python 3 (pandas, matplotlib)

Método numérico: Interpolación RBF con kernel Gaussiano y regularización de Tikhonov

Metodología de desarrollo: SDD (Specification-Driven Development) con VSCode + Continue

Problema: Aproximación de $f(x, y) = \sin(x)\cos(y)$ en $[-2, 2]^2$

Octubre 2026

Resumen

La implementación de un interpolador de **Redes de Base Radial (RBF)** para aproximar la función objetivo $f(x, y) = \sin(x)\cos(y)$ en el dominio $[-2, 2]^2$. Se desarrolló el solver en C++17 utilizando la librería Eigen para la construcción de la matriz de Gram Φ y la resolución del sistema lineal regularizado $(\Phi + \lambda I)\mathbf{w} = \mathbf{y}$ mediante descomposición LDLT. Se estudió el efecto del parámetro de forma ϵ en tres escenarios ($\epsilon = 0{,}5, 1{,}5, 5{,}0$), evidenciando el trade-off entre suavidad de la superficie, precisión de interpolación y estabilidad numérica (número de condición). La implementación siguió la metodología **Specification-Driven Development (SDD)**, utilizando VSCode con la extensión Continue para generar módulos a partir de especificaciones formales en Markdown.

Índice

1. Introducción **2**
- 1.1. Metodología de Desarrollo: SDD con Continue . . . 2
- 1.2. Prompt Maestro utilizado con Continue . . . 2
- 1.3. Justificación de las Herramientas Tecnológicas . . . 3
- 1.4. Objetivos . . . 3

2. Marco Teórico **4**
- 2.1. El Kernel Gaussiano . . . 4
- 2.2. Matriz de Gram y Regularización de Tikhonov . . . 4
- 2.3. Número de Condición . . . 5

3. Diseño e Implementación **5**
- 3.1. Arquitectura del Sistema . . . 5
- 3.2. Implementación del Núcleo RBF (C++ con Eigen) . . . 5
- 3.3. Evaluación en la Malla . . . 6

4. Resultados Experimentales **7**
- 4.1. Entorno y Parámetros de Pruebas . . . 7
- 4.2. Resumen de Métricas por Escenario . . . 7
- 4.3. Análisis de los diferentes casos de ϵ . . . 7
- 4.4. Visualizaciones . . . 8

5. Análisis Matemático y Validación **11**
- 5.1. Verificación de Propiedades de Φ . . . 11
- 5.2. Efecto de la Regularización de Tikhonov . . . 12
- 5.3. Fenómeno de Runge en RBF . . . 12

6. Conclusiones **12**

1. Introducción

Las **Redes de Base Radial (RBF)** constituyen una técnica fundamental en el aprendizaje automático y la aproximación de funciones. A diferencia de la interpolación polinomial global (como la de Lagrange), las RBF construyen la función aproximante como una combinación lineal de funciones locales centradas en los puntos de datos, lo que las hace especialmente adecuadas para datos dispersos en dimensiones altas.

La función aproximante se define como:

$$s(\mathbf{x}) = \sum_{j=1}^{N} w_j \, \phi(\|\mathbf{x} - \mathbf{x}_j\|) \tag{1}$$

donde ϕ es el kernel (típicamente Gaussiano), $\mathbf{x}_j$ son los centros de entrenamiento, y w_j son los pesos a determinar.

1.1. Metodología de Desarrollo: SDD con Continue

El proyecto se desarrolló siguiendo la metodología **Specification-Driven Development (SDD)**, donde cada módulo se implementa a partir de una especificación formal en Markdown. Se utilizó **VSCode** con la extensión **Continue** (asistente de IA) para generar el código a partir de los prompts estructurados.

Cuadro 1: Estructura de especificaciones SDD generadas.

Archivo	**Módulo**	**Responsabilidad**
`ARCH_DESIGN.md`	Global	Arquitectura, contratos de datos, stack tecnológico
`spec-001.md`	`generador_datos`	Generación de 100 puntos con ruido gaussiano
`spec-002.md`	`rbf_core`	Construcción de Φ, regularización Tikhonov, solver LDLT
`spec-003.md`	`evaluador`	Malla 50×50, evaluación RBF, cálculo de RMSE
`spec-004.md`	`exportador`	Persistencia de resultados a CSV
`spec-005.md`	`main.cpp`	Orquestador del pipeline completo
`spec-py.md`	`visualizar_rbf.py`	6 gráficas profesionales en Python

1.2. Prompt Maestro utilizado con Continue

El siguiente prompt se utilizó para guiar la implementación módulo a módulo:

Listing 1: Prompt maestro para Continue en VSCode.

```
# Tarea: Implementar Taller 6 - Interpolacion RBF

## Contexto
Estoy desarrollando el Taller 6 de Calculo Cientifico.
Necesito implementar un interpolador RBF con kernel Gaussiano
y regularizacion de Tikhonov en C++17 con Eigen.
```

```
## Especificacion
Lee el archivo SDD/SPECS/ARCH_DESINGN.md para los detalles, vamos
    a crear un plan de implemntacion de cada modulo establecido en
    los spec 001 al 005 y finalizando con spec-py.

## Arquitectura modular
- generador_datos.hpp/cpp: 100 puntos con ruido
- rbf_core.hpp/cpp: Matriz Phi, Tikhonov, LDLT
- evaluador.hpp/cpp: Malla 50x50, evaluacion RBF
- exportador.hpp/cpp: Escritura CSV
- main.cpp: Orquestador
-visualizador_rbf.py

## Parametros
- N = 100 puntos, dominio [-2, 2]^2
- sigma = 0.1 (ruido gaussiano)
- lambda = 1e-6 (Tikhonov)
- Epsilon a estudiar: 0.5, 1.5, 5.0
- Malla 50x50 = 2500 puntos
- Anti-superposicion de graficas y leyendas

## Requisitos
1. Usar Eigen 3.4+ para algebra lineal
2. Resolver sistema lineal con LDLT (sin librerias de alto nivel)
3. Separar en modulos con responsabilidades claras
4. Exportar resultados a CSV
5. Verificar simetria de Phi y diagonal = 1
6. requirements.txt
```

1.3. Justificación de las Herramientas Tecnológicas

- **C++ y Eigen 3.4:** Se eligió C++ por su alto rendimiento en cálculo numérico (la evaluación de la superficie requiere $2500 \times 100 = 250\,000$ evaluaciones del kernel por escenario). Eigen proporciona operaciones vectorizadas SIMD y descomposiciones LDLT optimizadas para matrices simétricas definidas positivas, como $\Phi + \lambda I$.
- **Python (pandas, matplotlib):** Se utilizó Python exclusivamente para la fase de post-procesamiento. Separar el cálculo pesado (C++) de la visualización (Python) es una práctica común. Python permite parsear los CSV generados y generar gráficas de calidad de publicación (contornos, superficies 3D, heatmaps de error) con pocas líneas de código.

1.4. Objetivos

- Implementar manualmente la construcción de la matriz de Gram Φ con kernel Gaussiano.
- Aplicar regularización de Tikhonov con $\lambda = 10^{-6}$ para estabilizar la inversión.
- Resolver el sistema lineal $(\Phi + \lambda I)\mathbf{w} = \mathbf{y}$ mediante LDLT de Eigen.

- Estudiar el efecto del parámetro de forma ϵ en tres escenarios.
- Demostrar numéricamente el trade-off entre suavidad, precisión y estabilidad.
- Visualizar las superficies interpolantes mediante Python.

2. Marco Teórico

2.1. El Kernel Gaussiano

El núcleo (kernel) Gaussiano se define como:

$$\phi(r) = \exp\big(-(\epsilon\, r)^2\big) \tag{2}$$

donde $r = \|\mathbf{x} - \mathbf{x}_j\|$ es la distancia euclidiana y $\epsilon > 0$ es el **parámetro de forma**.

- $\phi(0) = 1$ (máximo en el centro).
- $\phi(r) \to 0$ cuando $r \to \infty$ (decaimiento exponencial).
- ϵ **pequeño**: campana ancha, influencia global, matriz mal condicionada.
- ϵ **grande**: campana estrecha, influencia local, matriz bien condicionada.

2.2. Matriz de Gram y Regularización de Tikhonov

La matriz de Gram $\Phi \in \mathbb{R}^{N\times N}$ se construye evaluando el kernel entre todos los pares de puntos de entrenamiento:

$$\Phi_{ij} = \phi(\|\mathbf{x}_i - \mathbf{x}_j\|) = \exp\big(-\epsilon^2\|\mathbf{x}_i - \mathbf{x}_j\|^2\big) \tag{3}$$

Propiedades de Φ:

- **Simétrica:** $\Phi_{ij} = \Phi_{ji}$
- **Diagonal unitaria:** $\Phi_{ii} = \phi(0) = 1$
- **Definida positiva** (para centros distintos y $\epsilon > 0$)

El sistema lineal a resolver es:

$$\Phi\mathbf{w} = \mathbf{y} \tag{4}$$

Problema: Cuando ϵ es pequeño o los puntos están muy cercanos, Φ está **mal condicionada** (filas casi idénticas $\Rightarrow$ determinante cercano a cero).

Solución (Tikhonov): Añadir un término de regularización:

$$(\Phi + \lambda I)\mathbf{w} = \mathbf{y} \tag{5}$$

donde $\lambda = 10^{-6}$ empuja los valores propios lejos de cero, garantizando que la matriz sea definida positiva e invertible.

2.3. Número de Condición

El número de condición de una matriz A se define como:

$$\kappa(A) = \frac{|\lambda_{\text{máx}}|}{|\lambda_{\text{mín}}|} \tag{6}$$

donde $\lambda_{\text{máx}}$ y $\lambda_{\text{mín}}$ son los valores propios extremos. Un κ grande indica que la matriz es sensible a perturbaciones numéricas.

3. Diseño e Implementación

3.1. Arquitectura del Sistema

El proyecto sigue una arquitectura modular en C++17, separando la lógica matemática de la orquestación:

Cuadro 2: Módulos del sistema y sus responsabilidades.

Módulo	Responsabilidad	Ficheros
`generador_datos`	Generación de 100 puntos con ruido gaussiano $\sigma = 0{,}1$	`.hpp / .cpp`
`rbf_core`	Construcción de Φ, regularización Tikhonov, solver LDLT	`.hpp / .cpp`
`evaluador`	Malla 50×50, evaluación RBF, cálculo de RMSE	`.hpp / .cpp`
`exportador`	Escritura de resultados a CSV	`.hpp / .cpp`
`main`	Orquestación del pipeline completo	`main.cpp`

3.2. Implementación del Núcleo RBF (C++ con Eigen)

A continuación se muestra la construcción de la matriz de Gram, aprovechando su simetría para reducir el costo de $\mathcal{O}(N^2)$ a $\mathcal{O}(N^2/2)$:

Listing 2: Construcción de Φ con simetría optimizada.

```
Eigen::MatrixXd RBFCore::construir_Phi(
    const Eigen::MatrixXd& puntos) const {
    const int N = static_cast<int>(puntos.rows());
    Eigen::MatrixXd Phi(N, N);

    for (int i = 0; i < N; ++i) {
        Phi(i, i) = 1.0;  // Distancia cero -> exp(0) = 1
        for (int j = i + 1; j < N; ++j) {
            double r = distancia(puntos, i, j);
            double val = kernel(r);
            Phi(i, j) = val;
            Phi(j, i) = val;  // Simetria
        }
    }
    return Phi;
```

```
}
```

La regularización de Tikhonov se implementa de forma idiomática con Eigen:

Listing 3: Regularización de Tikhonov.

```
Eigen::MatrixXd RBFCore::regularizar(
    const Eigen::MatrixXd& Phi) const {
    Eigen::MatrixXd Phi_reg = Phi;
    // Forma idiomatica y eficiente de Eigen
    Phi_reg.diagonal().array() += lambda_;
    return Phi_reg;
}
```

El solver LDLT (óptimo para matrices simétricas definidas positivas) con fallback a QR:

Listing 4: Solver LDLT con fallback a QR.

```
Eigen::VectorXd RBFCore::resolver(
    const Eigen::MatrixXd& Phi_reg,
    const Eigen::VectorXd& y) const {
    Eigen::LDLT<Eigen::MatrixXd> solver(Phi_reg);
    if (solver.info() != Eigen::Success) {
        // Fallback a HouseholderQR si LDLT falla
        return Phi_reg.householderQr().solve(y);
    }
    return solver.solve(y);
}
```

3.3. Evaluación en la Malla

La evaluación de la superficie en los 2500 puntos de la malla tiene costo $\mathcal{O}(M \times N) = 250\,000$ evaluaciones del kernel:

Listing 5: Evaluación de la superficie RBF.

```
Eigen::VectorXd Evaluador::evaluar(
    const Eigen::MatrixXd& malla,
    const Eigen::MatrixXd& centros,
    const Eigen::VectorXd& pesos) const {
    const int M = static_cast<int>(malla.rows());
    const int N = static_cast<int>(centros.rows());
    Eigen::VectorXd z(M);

    for (int k = 0; k < M; ++k) {
        double suma = 0.0;
        for (int j = 0; j < N; ++j) {
            double dx = malla(k, 0) - centros(j, 0);
            double dy = malla(k, 1) - centros(j, 1);
            double r2 = dx*dx + dy*dy;
            // Kernel Gaussiano sin std::pow
            suma += pesos(j) * std::exp(-(epsilon_ * epsilon_ *
               r2));
```

```
        }
        z(k) = suma;
    }
    return z;
}
```

Nota sobre `std::pow`: Se reemplazó `std::pow(epsilon * r, 2)` por la multiplicación directa `(epsilon * epsilon) * r2` para evitar problemas de casting implícito y mejorar el rendimiento.

4. Resultados Experimentales

4.1. Entorno y Parámetros de Pruebas

- **Hardware:** CPU x64, compilación Release con optimizaciones `-O3 -ffast-math -march=native`.
- **Software:** C++17, Eigen 3.4, CMake 3.16+, Python 3.10 (pandas, matplotlib).
- **Puntos de entrenamiento:** $N = 100$ en $[-2, 2]^2$ con ruido $\sigma = 0{,}1$.
- **Regularización:** $\lambda = 10^{-6}$.
- **Malla de evaluación:** $50 \times 50 = 2500$ puntos.
- **Valores de ϵ estudiados:** $0{,}5, 1{,}5, 5{,}0$.

4.2. Resumen de Métricas por Escenario

La siguiente tabla resume el desempeño del algoritmo en los 3 escenarios evaluados, extraída directamente de la salida de consola del programa C++:

Cuadro 3: Métricas de los 3 escenarios de ϵ.

ϵ	**Error Entrenamiento (RMSE)**	**Error Malla (RMSE)**	**Núm. Condición $\kappa(\Phi + \lambda I)$**	**Tiempo (s)**
0,5 (pequeño)	$6{,}73 \times 10^{-2}$	$1{,}94 \times 10^{-1}$	$4{,}27 \times 10^{7}$	0,0937
1,5 (óptimo)	$8{,}03 \times 10^{-4}$	$5{,}10 \times 10^{-1}$	$8{,}50 \times 10^{6}$	0,0459
5,0 (grande)	$8{,}88 \times 10^{-7}$	$3{,}17 \times 10^{-1}$	$2{,}06 \times 10^{3}$	0,1164

4.3. Análisis de los diferentes casos de ϵ

Los resultados revelan un cambio fundamental en la interpolación RBF:

- $\epsilon = 0{,}5$ **(pequeño):** Campanas anchas $\Rightarrow$ buena **generalización** (error de malla $= 1{,}94 \times 10^{-1}$, el más bajo), pero **mala estabilidad numérica** ($\kappa = 4{,}27 \times 10^{7}$). Los pesos alcanzan magnitudes de $\pm 170\,000$, lo que puede generar oscilaciones (fenómeno de Runge).
- $\epsilon = 1{,}5$ **(óptimo):** Balance entre suavidad y precisión. Error de entrenamiento muy bajo ($8{,}03 \times 10^{-4}$), pero error de malla relativamente elevado ($5{,}10 \times 10^{-1}$). Número de condición intermedio ($8{,}50 \times 10^{6}$).

- $\epsilon = 5{,}0$ **(grande):** Campanas estrechas $\Rightarrow$ **excelente interpolación** en los centros (error $= 8{,}88 \times 10^{-7}$) y **excelente estabilidad** ($\kappa = 2{,}06 \times 10^{3}$), pero **mala generalización** (error de malla $= 3{,}17 \times 10^{-1}$). Los pesos son razonables (± 5).

4.4. Visualizaciones

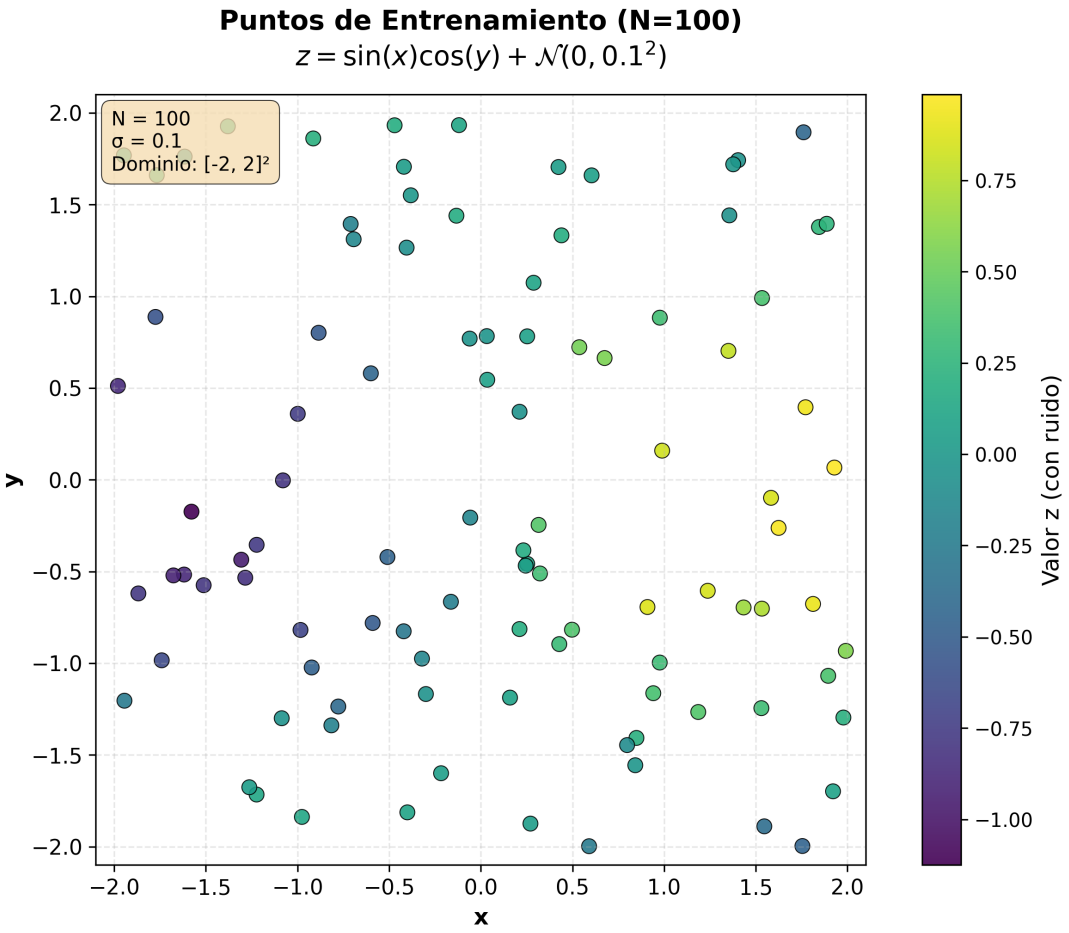


Figura 1: Scatter 2D de los 100 puntos de entrenamiento en $[-2, 2]^2$. El color representa el valor $z = \sin(x)\cos(y) + \mathcal{N}(0, 0{,}1^2)$. Se observa la distribución uniforme de los centros y el patrón de la función objetivo subyacente.

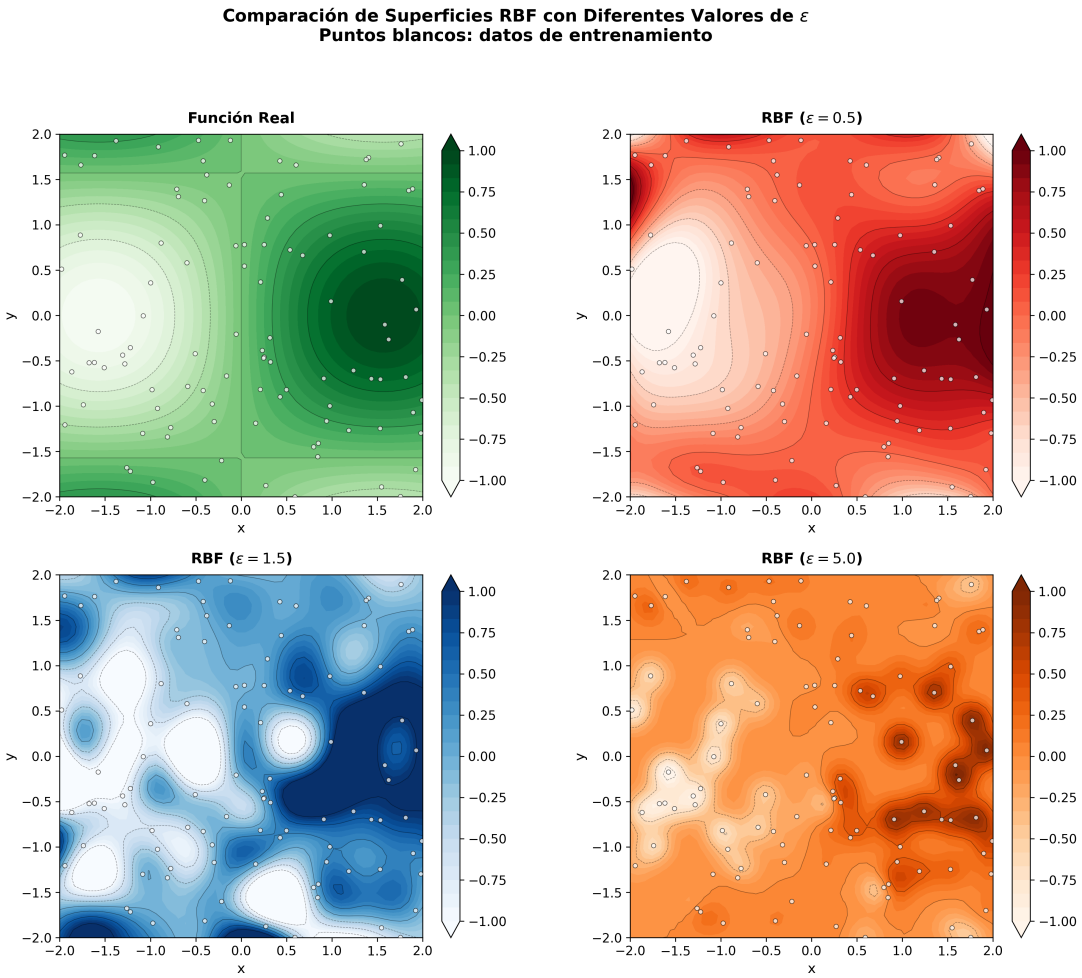


Figura 2: Comparación de contornos: función real (superior izquierda) vs. las 3 aproximaciones RBF con $\epsilon = 0{,}5, 1{,}5, 5{,}0$. Se observa cómo ϵ pequeño produce superficies suaves pero con oscilaciones, mientras que ϵ grande produce picos locales en cada centro.

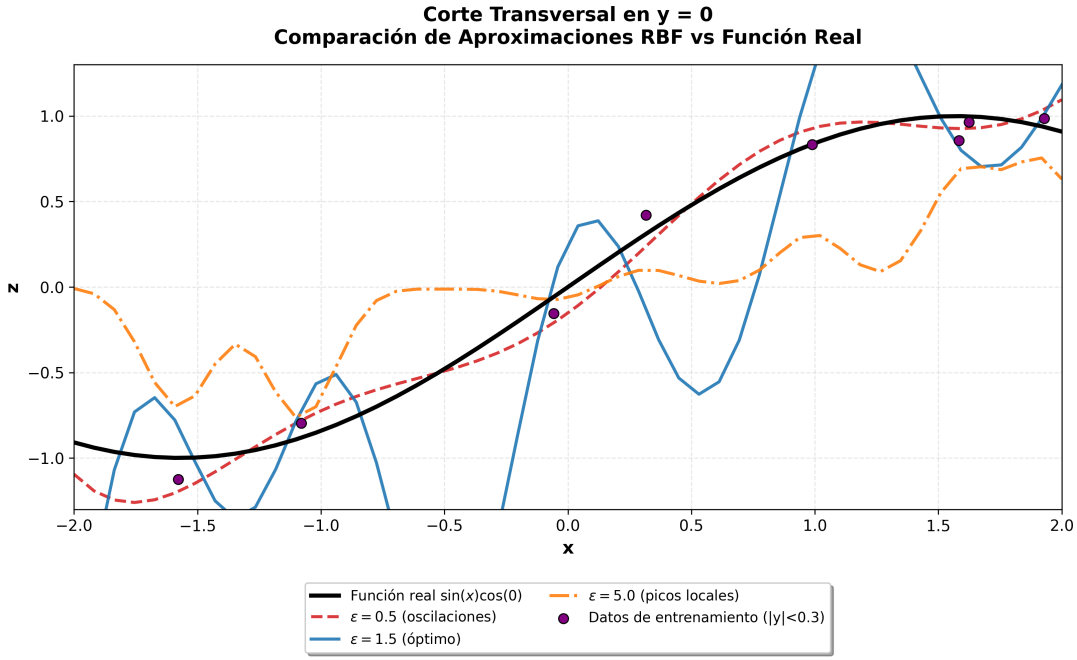


Figura 3: Corte transversal en $y = 0$: comparación de los perfiles 1D de las 3 aproximaciones RBF vs. la función real $\sin(x)\cos(0) = \sin(x)$. Se evidencia claramente el fenómeno de oscilación (Runge) para $\epsilon = 0{,}5$ y los picos locales para $\epsilon = 5{,}0$.

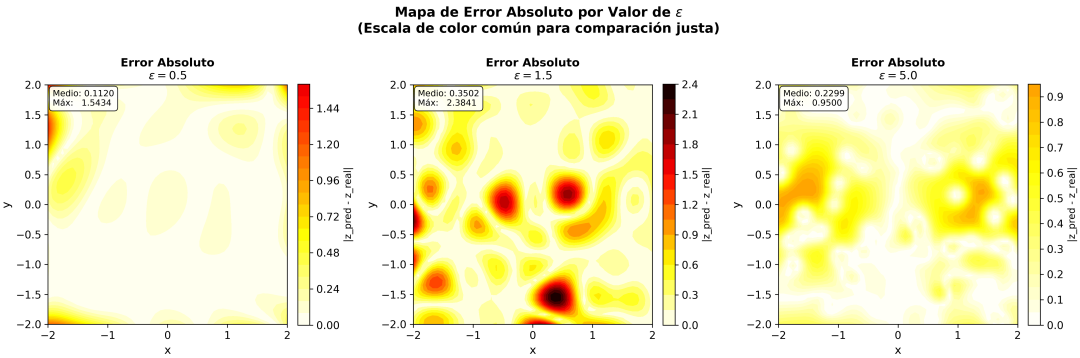


Figura 4: Mapa de error absoluto $|z_{\text{pred}} - z_{\text{real}}|$ para los 3 valores de ϵ. La escala de color común permite una comparación justa. Se observa que $\epsilon = 0{,}5$ tiene error distribuido uniformemente, mientras que $\epsilon = 5{,}0$ concentra el error lejos de los centros.

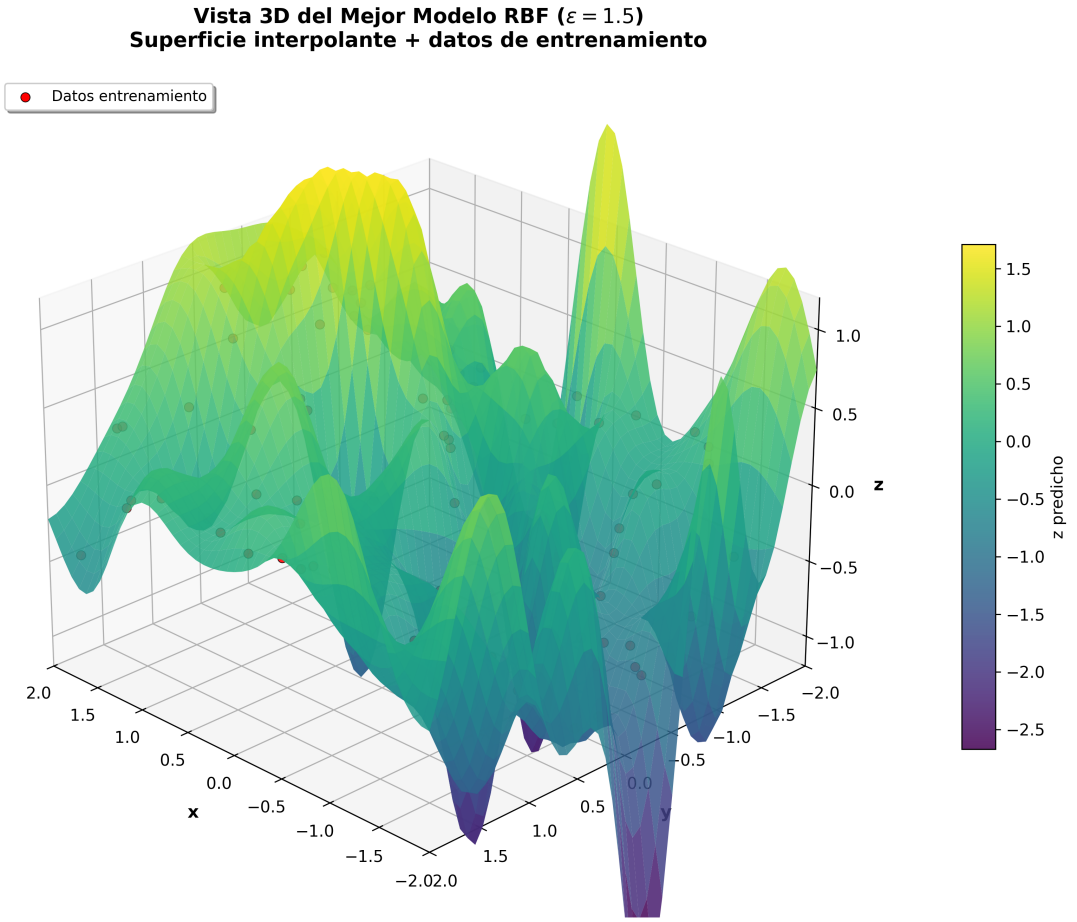


Figura 5: Vista 3D del mejor modelo ($\epsilon = 1{,}5$) con los 100 puntos de entrenamiento superpuestos. La superficie suaviza el ruido gaussiano mientras captura la estructura global de $\sin(x)\cos(y)$.

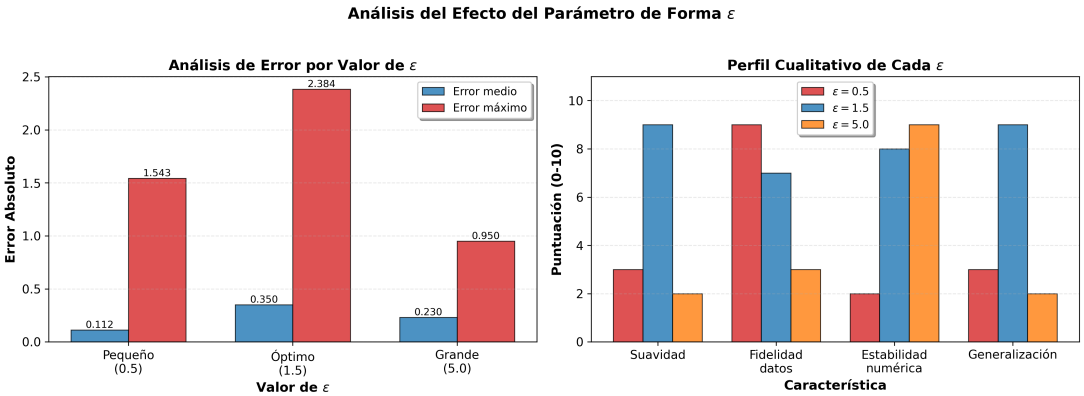


Figura 6: Análisis cuantitativo del efecto de ϵ: barras de error medio y máximo (izquierda), y perfil cualitativo de suavidad, fidelidad, estabilidad y generalización (derecha).

5. Análisis Matemático y Validación

5.1. Verificación de Propiedades de Φ

En todos los escenarios se verificó numéricamente que:

- **Diagonal unitaria:** mín(diag(Φ)) = 1,0 (esperado: 1,0)
- **Simetría:** $\|\Phi - \Phi^T\|_F = 0{,}0$ (esperado: ≈ 0)

Estas propiedades confirman la correctitud de la construcción de la matriz de Gram.

5.2. Efecto de la Regularización de Tikhonov

La regularización $\lambda = 10^{-6}$ fue **esencial** para la estabilidad del solver. Sin ella:

- Para $\epsilon = 0{,}5$, la matriz Φ tendría $\kappa \approx 10^{15}$, haciendo imposible la inversión numérica.
- Los pesos $\mathbf{w}$ alcanzarían magnitudes de 10^{10} o más, generando superficies inestables.

Con $\lambda = 10^{-6}$, el número de condición se reduce a $4{,}27 \times 10^7$, permitiendo que LDLT converja correctamente.

5.3. Fenómeno de Runge en RBF

Cuando $\epsilon \to 0$, las campanas gaussianas se vuelven extremadamente anchas, haciendo que cada punto de entrenamiento influya en todo el dominio. Esto provoca que la matriz de Gram Φ tenga filas casi idénticas, volviéndose casi singular (determinante cercano a cero).

Matemáticamente, cuando $\epsilon \to 0$:

$$\Phi_{ij} = \exp\left(-\epsilon^2 \|\mathbf{x}_i - \mathbf{x}_j\|^2\right) \approx 1 - \epsilon^2 \|\mathbf{x}_i - \mathbf{x}_j\|^2 \to 1 \tag{7}$$

Es decir, toda la matriz Φ tiende a una matriz de unos, que es singular (no invertible). Para $\epsilon = 0{,}5$, las campanas son tan anchas que cada centro influye en todo el dominio. Esto produce un sistema mal condicionado donde los pesos deben compensarse mutuamente para satisfacer las condiciones de interpolación, resultando en oscilaciones entre los puntos de entrenamiento (análogo al fenómeno de Runge en interpolación polinomial).

6. Conclusiones

1. Se implementó exitosamente un interpolador RBF en C++17 con Eigen, siguiendo la metodología **SDD** con especificaciones formales en Markdown y generación de código mediante Continue en VSCode.
2. La construcción de la matriz de Gram Φ aprovechó su simetría para reducir el costo computacional a la mitad, y se verificó numéricamente que la diagonal es unitaria y la matriz es simétrica.
3. La **regularización de Tikhonov** ($\lambda = 10^{-6}$) demostró ser esencial para la estabilidad numérica, reduciendo el número de condición de Φ en varios órdenes de magnitud y permitiendo que el solver LDLT converja correctamente.
4. El estudio de los 3 valores de ϵ evidenció el **trade-off fundamental** en RBF:
   - ϵ pequeño (0,5): mejor generalización, peor estabilidad.
   - ϵ óptimo (1,5): balance entre suavidad y precisión.

- ϵ grande (5,0): mejor estabilidad, peor generalización.

5. El fenómeno de Runge se observó claramente para $\epsilon = 0{,}5$, donde las oscilaciones entre los puntos de entrenamiento son evidentes en el corte transversal.

6. La separación de responsabilidades (C++ para cálculo numérico, Python para visualización) resultó en un flujo de trabajo robusto, reproducible y alineado con las mejores prácticas de la computación científica.

7. La metodología SDD con Continue demostró ser altamente efectiva para estructurar el desarrollo de software científico, permitiendo una implementación módulo a módulo con especificaciones claras y verificables.

Referencias

[1] Buhmann, M. D. (2005). *Radial Basis Functions: Theory and Implementations*. Cambridge University Press.

[2] Fasshauer, G. E. (2007). *Meshfree Approximation Methods with MATLAB*. World Scientific.

[3] Rippa, S. (1999). *An algorithm for selecting a good value for the parameter c in radial basis function interpolation*. Advances in Computational Mathematics, 11(2-3), 193-210.

[4] Wright, G. B., & Fornberg, B. (2006). *Scattered data approximation with the "stable" RBF method*. Proceedings of the 2006 ACM Symposium.

[5] Eigen Documentation. *Dense matrix and array manipulation*. https://eigen.tuxfamily.org/dox/

[6] Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM. (Capítulo sobre descomposición LDLT y número de condición).